

Resolución: Colocación de Cámaras de Vigilancia por Ramificación y Poda

Este documento presenta el diseño de un algoritmo de **Ramificación y Poda** (*Branch and Bound*) para resolver el problema de encontrar el número mínimo de cámaras de vigilancia necesarias para cubrir todas las calles del campus Río Ebro. Este problema es equivalente al problema de optimización NP-difícil conocido como **Cobertura de Vértices Mínima** (*Minimum Vertex Cover*).

a) Representación de Soluciones y Árbol de Búsqueda

1. Representación de las Soluciones

Consideramos el campus como un grafo no dirigido $G = (V, E)$, donde:

- $V = \{1, 2, \dots, n\}$ es el conjunto de intersecciones (vértices).
- $E = \{(u, v) \mid u, v \in V\}$ es el conjunto de calles que conectan las intersecciones (aristas).

Representamos cualquier solución candidata mediante un vector binario de decisión de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i indica si se coloca una cámara en el vértice i :

- $x_i = 1$: Se coloca una cámara en el vértice i .
- $x_i = 0$: No se coloca una cámara en el vértice i .

Restricción Implícita (Cobertura Completa)

Para que la solución sea válida, toda calle $(u, v) \in E$ debe estar vigilada por al menos una cámara. Es decir, se debe cumplir que:

$$x_u + x_v \geq 1 \quad \forall (u, v) \in E$$

2. Descripción del Árbol de Espacio de Soluciones

El espacio de búsqueda se representa mediante un **árbol de decisión binario**:

- **Raíz (Nivel 0)**: Ningún vértice ha sido evaluado (estado inicial).
- **Nivel i ($1 \leq i \leq n$)**: Representa la toma de decisión para el vértice i (si colocar cámara o no).
- **Ramificación**: De cada nodo en el nivel $i - 1$ nacen dos ramas
- **Hojas (Nivel n)**: Representan asignaciones completas de cámaras para todos los vértices.

b) Función de Coste Real $c(x)$ a Minimizar

Para cualquier nodo x en el árbol de búsqueda (donde ya se han tomado decisiones para los primeros k vértices de la tupla):

1. **Coste Acumulado Real $g(x)$** : Es el número de cámaras que ya se han colocado de manera efectiva en el camino desde la raíz hasta el nodo x :

$$g(x) = \sum_{i=1}^k x_i$$

2. **Función de Coste Real de una Solución Completa $c(x)$** : Si un nodo x es una hoja (nivel n), su coste final se evalúa como:

$$c(x) = \begin{cases} \sum_{i=1}^n x_i & \text{si } X \text{ es una cobertura válida} \\ \infty & \text{si } X \text{ es inviable (deja calles sin vigilar)} \end{cases}$$

El objetivo del algoritmo es encontrar una hoja viable x que minimice $c(x)$.

c) Función de Coste Estimado $\hat{c}(x)$ para la Ramificación

En la estrategia de Ramificación y Poda, para priorizar la exploración de nodos prometedores y podar ramas inútiles, calculamos una **cota inferior** $\hat{c}(x)$ de cualquier hoja descendiente de x .

Sea $E_{\text{no_vig}}$ el conjunto de calles (aristas) que **todavía no están vigiladas** por ninguna de las cámaras ya colocadas en los primeros k vértices:

$$E_{\text{no_vig}} = \{(u, v) \in E \mid u > k \wedge v > k\}$$

Para vigilar las aristas que quedan en $E_{\text{no_vig}}$, obligatoriamente tendremos que añadir más cámaras. Definimos $\hat{c}(x)$ como:

$$\hat{c}(x) = g(x) + \text{Estimación_Inferior}(E_{\text{no_vig}})$$

Para que el algoritmo sea óptimo y admisible, la estimación debe ser una cota inferior segura (nunca sobreestimar el coste real). Proponemos dos opciones válidas:

Heurística de Grado Máximo

Sea Δ_{rest} el grado máximo de los vértices restantes (es decir, el máximo número de calles sin vigilar que convergen en una sola intersección disponible). Una cámara adicional puede cubrir, como máximo, Δ_{rest} aristas. Por tanto, el número mínimo absoluto de cámaras adicionales necesarias es:

$$\hat{c}(x) = g(x) + \left\lceil \frac{|E_{\text{no_vig}}|}{\Delta_{\text{rest}}} \right\rceil$$

d) Función de Poda $U(x)$ (Cota Superior)

La función de poda $U(x)$ representa una **cota superior** de la solución óptima. Se calcula completando la solución parcial del nodo x utilizando un algoritmo voraz (*greedy*) rápido para obtener una solución factible:

Algoritmo Voraz para calcular $U(x)$

1. Inicializamos la solución factible temporal con las decisiones ya tomadas hasta el nivel k y el conjunto de aristas no vigiladas $E_{\text{temp}} \leftarrow E_{\text{no_vig}}$.
2. Mientras E_{temp} no esté vacío:
 - Seleccionamos el vértice restante $v > k$ que tenga el mayor número de aristas incidentes en E_{temp} (criterio de grado máximo).
 - Colocamos una cámara en v (es decir, hacemos $x_v = 1$ en nuestra solución temporal).
 - Eliminamos de E_{temp} todas las aristas cubiertas por v .
3. Sea C_{voraz} el número total de cámaras añadidas mediante este proceso.

Definimos la cota superior del nodo x como:

$$U(x) = g(x) + C_{\text{voraz}}$$

Criterio de Poda

Durante la ejecución de la búsqueda (por ejemplo, mediante exploración por *Best-First* utilizando una cola de prioridad basada en $\hat{c}(x)$):

1. Mantenemos una variable global Cota_Global (el menor valor de $U(x)$ encontrado hasta el momento para cualquier solución factible completa).
2. Para cada nuevo nodo generado y :
 - Si su coste estimado $\hat{c}(y) \geq \text{Cota_Global}$, el nodo y **se poda de inmediato** (se descarta de la búsqueda), ya que matemáticamente es imposible obtener una solución mejor a través de esa rama.